CAD for VLSI Design
Programming Assignment 4

Analog Floorplan Project Report

Advisor: Yu-Guang Chen

Student Info: 電機 4A

Student ID: 110501521

Student Name: 徐松廷

1. How to Compile and Execute My Program

a. My Makefile

依照助教要求，本次 PA4 會需要按照以下方式，利用 makefile 支援 make all、make run、make clean，編譯.cpp file 並執行.exe file，並接收使用者的 input file name 以及 output file name 執行.exe，最終產生 partition 的結果，再利用 make clean 清除執行檔。

```
CXX = g++
CXXFLAGS = -std=c++11 -Wall -Wextra

# 檔名與資料夾
TARGET = 110501521_PA4
SRC = src/Parser.cpp
INC = -Iinc

# 所有物件檔
OBJS = $(TARGET).o $(SRC:.cpp=.o)

# 預設目標
all: $(TARGET)

# 編譯主程式與 Parser
$(TARGET): $(OBJS)
	$(CXX) $(CXXFLAGS) -o $@ $(OBJS)
```

```
$(TARGET).o: $(TARGET).cpp
	$(CXX) $(CXXFLAGS) $(INC) -c $< -o $@

# 編譯 Parser.cpp
src/Parser.o: src/Parser.cpp inc/Parser.h
	$(CXX) $(CXXFLAGS) $(INC) -c $< -o $@

# 執行程式
run: $(TARGET)
	@if [ -z "$(input)" ] || [ -z "$(output)" ]; then \
		echo "請使用：make run input=xxx output=yyy"; \
	else \
		./$(TARGET) $(input) $(output); \
	fi

# 清除可執行檔與中間產物
clean:
	rm -f $(TARGET) *.o src/*.o
```

b. Compiling and Executing Steps:

依照助教要求，本次 PA2 會需要按照以下方式，利用 makefile 去編譯.cpp file 並執行.exe file，產生.out 後利用助教提供的 checker 確認我們的 Partition 結果是否正確，詳細步驟如下:

I. Step 1: make all

```
[s110501521@ncuee 110501521_PA4]$ make all
```

II. **Step 2: make run input=case.block output=case.out**

```
[s110501521@ncuee 110501521_PA4]$ make run input=case1.block output=case1.out
```

III. **Step 3: ./PA4_checker case1.block case1.out**

```
[s110501521@ncuee 110501521_PA4]$ ./PA4_checker case1.block case1.out
```

IV. **Step 4: make clean**

```
[s110501521@ncuee 110501521_PA4]$ make clean
```

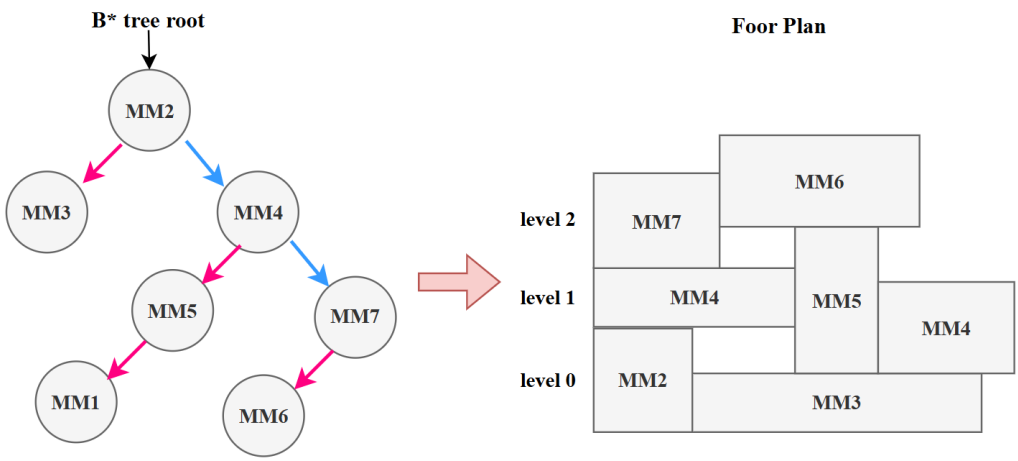
V. **Step 5: tar -czvf 110501521_PA4.tar 110501521_PA4**

```
[s110501521@ncuee 110501521_PA4]$ tar -czvf 110501521_PA4.tar 110501521_PA
```

2. The Completion of The Assignment

a. Floor Planning and Block Position/Area Calculation

這次的作業要求我們根據助教提供的每個 Block 的尺寸，完成一個 Floorplan。也就是將這些 Block 以合理的順序排列，使其整齊地擺放在版圖中。在擺放的過程中，我們需要計算每個 Block 的具體位置，並統整整體的使用面積（Total Area）。目前我的程式已能準確地完成這些需求，包括初始化的放置結果，也可以成功輸出對應數據:




```
451.2480
23.80 18.96
33.61
MM0 0.00 0.00 (1.90 5.54 8 1)
MM1 19.00 0.00 (2.90 9.54 8 1)
MM2 17.10 0.00 (1.90 4.74 1 1)
MM3 3.80 5.54 (7.90 6.74 1 1)
MM4 21.90 0.00 (1.90 18.96 1 4)
MM5 9.50 0.00 (1.90 4.74 4 1)
MM6 0.00 5.54 (1.90 4.24 2 1)
MM9 11.70 4.24 (1.90 4.24 2 1)
MM10 3.80 0.00 (1.90 5.54 10 1)
MM11 7.60 0.00 (1.90 5.54 2 1)
MM12 1.90 0.00 (1.90 5.54 2 1)
MM13 1.90 5.54 (1.90 5.54 2 1)
MM14 13.30 0.00 (1.90 4.24 2 1)
MM15 5.70 0.00 (1.90 4.24 2 1)
MM21 11.40 0.00 (1.90 4.24 2 1)
MM24 15.20 0.00 (1.90 4.24 2 1)
```

b. INL Calculation

$$S_{actual}(n) = \sum_{k=1}^{n} d_k^2$$

$$S_{ideal}(n) \approx \boldsymbol{an + b}$$

$$\boldsymbol{INL = max}\,|\,S_{actual}(n) - S_{ideal}(n)\,|$$

在這次的作業中我們還需要根據 Bounding Box 的中心座標，計算每個 Block 到該中心點的距離，並據此評估 INL（Integrated Non-Linearity）指標。我們採用課堂中定義的方法來計算 INL，透過累積距離平方值的方式生成實際曲線（S_actual），再透過線性回歸擬合出理想曲線（S_ideal），作為比較的基準，目前我們的程式已能準確完成這些計算，並正確評估 INL 值:

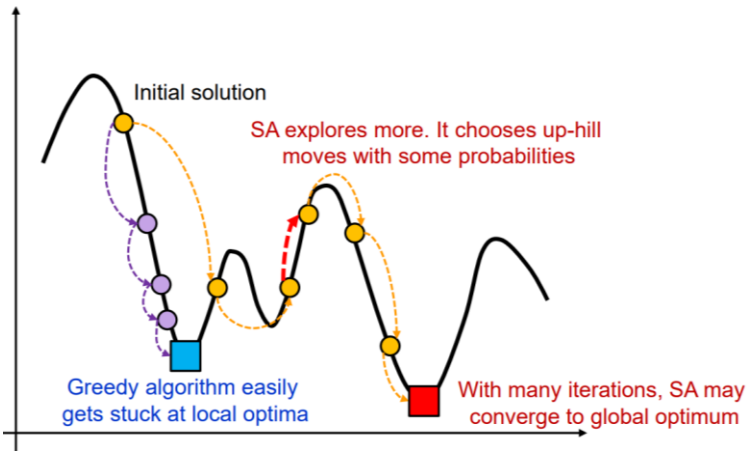
```
Algorithm Compute_S_actual
Input: placement P (includes node_vec, centroid coordinates Xc, Yc)
Output: Vector S_actual, where S_actual[i] is cumulative squared distance to centroid of first i+1 blocks

sorted_nodes ← SortNodesByID(P.node_vec)
Initialize S_actual as an array of size N = |sorted_nodes|
S_actual[0] ← (x0 - Xc)² + (y0 - Yc)², where (x0, y0) is the center of sorted_nodes[0]

for i = 1 to N-1 do
    (xi, yi) ← center of sorted_nodes[i]
    S_actual[i] ← S_actual[i−1] + (xi − Xc)² + (yi − Yc)²

P.S_actual ← S_actual
```

```
Algorithm Compute_S_ideal
Input: placement P (includes S_actual)
Output: Vector S_ideal, representing a best-fit line approximating S_actual

N ← length(P.S_actual)
Initialize: sum_x ← 0, sum_y ← 0, sum_x_sqr ← 0, sum_xy ← 0

for i = 0 to N − 1 do
    sum_x     ← sum_x     + i
    sum_y     ← sum_y     + P.S_actual[i]
    sum_x_sqr ← sum_x_sqr + i²
    sum_xy    ← sum_xy    + i × P.S_actual[i]

a ← (N × sum_xy − sum_x × sum_y) / (N × sum_x_sqr − sum_x²)   ← 斜率
b ← (sum_y − a × sum_x) / N                                   ← 截距

Initialize S_ideal as an array of size N
for i = 0 to N − 1 do
    P.S_ideal[i] ← a × i + b
```

```
451.2480
23.80 18.96
33.61
MM0 0.00 0.00 (1.90 5.54 8 1)
MM1 19.00 0.00 (2.90 9.54 8 1)
MM2 17.10 0.00 (1.90 4.74 1 1)
MM3 3.80 5.54 (7.90 6.74 1 1)
MM4 21.90 0.00 (1.90 18.96 1 4)
MM5 9.50 0.00 (1.90 4.74 4 1)
MM6 0.00 5.54 (1.90 4.24 2 1)
MM9 11.70 4.24 (1.90 4.24 2 1)
MM10 3.80 0.00 (1.90 5.54 10 1)
MM11 7.60 0.00 (1.90 5.54 2 1)
MM12 1.90 0.00 (1.90 5.54 2 1)
MM13 1.90 5.54 (1.90 5.54 2 1)
MM14 13.30 0.00 (1.90 4.24 2 1)
MM15 5.70 0.00 (1.90 4.24 2 1)
MM21 11.40 0.00 (1.90 4.24 2 1)
MM24 15.20 0.00 (1.90 4.24 2 1)
```

c. Perturbation with Simulated Annealing

```
Input: Initial placement P0, output file name
Output: Optimized placement P_best

P_current ← deep copy of P0
temperature ← 1000.0
cooling_rate ← 0.99

for iteration = 1 to 10000 do
    P_new ← deep copy of P_current

    // Perturb the placement
    With 80% probability: swap two adjacent blocks in P_new.node_vec
    With 20% probability: randomly change aspect ratio of a block

    // Rebuild floorplan and compute metrics
    Clean placement data in P_new
    P_new.root ← ConstructTree(P_new.node_vec, ...)
    ComputeCentroid(P_new)
    ComputeInterconnectLength(P_new)
    P_new.cost ← EvaluateCost(P_new)

    // Accept or reject new placement
    Δ ← P_new.cost − P_current.cost
    if Δ < 0 or exp(−Δ / temperature) > random(0, 1) then
        P_current ← deep copy of P_new

    temperature ← temperature × cooling_rate

    if iteration mod 500 = 0 then
        WritePlacementToFile(P_current, output file name)
return P_current
```



在本次作業中，除了使用 B*-Tree 結構來生成初始的 Floorplan 之外，我也實作了 Simulated Annealing 演算法，並搭配多種 Perturbation Strategies，以期在搜尋過程中找出具有更低 Cost 的 Floor plan 結果。其詳細的演算法流程如上述虛擬碼所示。根據實驗結果顯示，Simulated Annealing 確實能有效改善最終的解，顯著降低 Cost，整體優化效果良好。

d. Write File

在寫檔的部分，將所有 Block 的實際擺放位置與相關資訊輸出至檔案中。包含了整體排版所對應的總面積、每個 Block 的左下角座標、長寬資訊，以及對應的擺放方向，我們也會一併計算並輸出 INL，整體輸出的結果將會儲存在指定的 output 檔案中:

```
Input: placement P, output filename F
Output: Writes placement result to file F

Open file F for writing as outFile
if outFile fails to open then
    Print "can't open F"
    return

Let W ← P.right_bound        ← Final layout width
Let H ← P.up_bound           ← Final layout height
Write total area A = W × H to outFile (4 decimal places)
Write W and H to outFile (2 decimal places)
Write P.INL to outFile (2 decimal places)

Let sorted_nodes ← nodes in P.node_vec sorted by ID
for each node n in sorted_nodes do
    Let info ← n.candidate[n.candidate_idx]  ← Final size/configuration of the block
    Write to outFile:
    n.name, n.x, n.y,
    (info.width, info.height, info.col_multiple, info.row_multiple)
```

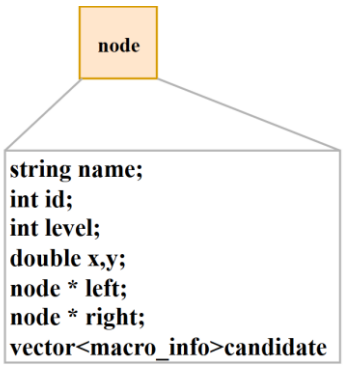
```
451.2480
23.80 18.96
33.61
MM0 0.00 0.00 (1.90 5.54 8 1)
MM1 19.00 0.00 (2.90 9.54 8 1)
MM2 17.10 0.00 (1.90 4.74 1 1)
MM3 3.80 5.54 (7.90 6.74 1 1)
MM4 21.90 0.00 (1.90 18.96 1 4)
MM5 9.50 0.00 (1.90 4.74 4 1)
MM6 0.00 5.54 (1.90 4.24 2 1)
MM9 11.70 4.24 (1.90 4.24 2 1)
MM10 3.80 0.00 (1.90 5.54 10 1)
MM11 7.60 0.00 (1.90 5.54 2 1)
MM12 1.90 0.00 (1.90 5.54 2 1)
MM13 1.90 5.54 (1.90 5.54 2 1)
MM14 13.30 0.00 (1.90 4.24 2 1)
MM15 5.70 0.00 (1.90 4.24 2 1)
MM21 11.40 0.00 (1.90 4.24 2 1)
MM24 15.20 0.00 (1.90 4.24 2 1)
```

3. Method of My Design- Data Structure

a. struct node

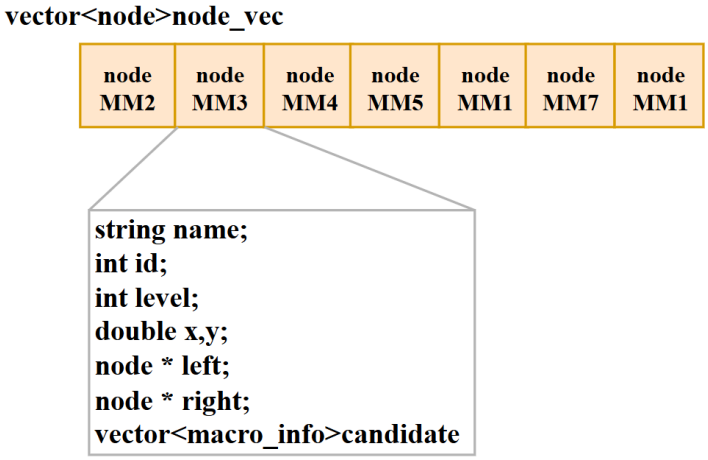
一開始我們會先定義一個叫做 node 的結構，來記錄每一個 block 的資料。像是 block 的名字、ID、它現在在第幾層（level），還有它左下角的位置、執行座標等等，這個結構也會記錄它在 B*-Tree 裡的 left child 跟 right child，另外還會把這個 block 所有可能的長寬組合都存在一個 list 裡，讓我們在擺放或調整的時候可以去選擇最合適的尺寸。

```
struct node{
    string name;
    int id;
    int level;        //目前node被擺在第幾層(從第0層開始)
    double x,y;       //一個block左下角的座標
    double xk,yk;     //一個block的質心座標
    node* left;       //left child，右邊的cell
    node* right;      //right child，上方的cell
    int candidate_idx;
    vector<macro_info>candidate;
};
```



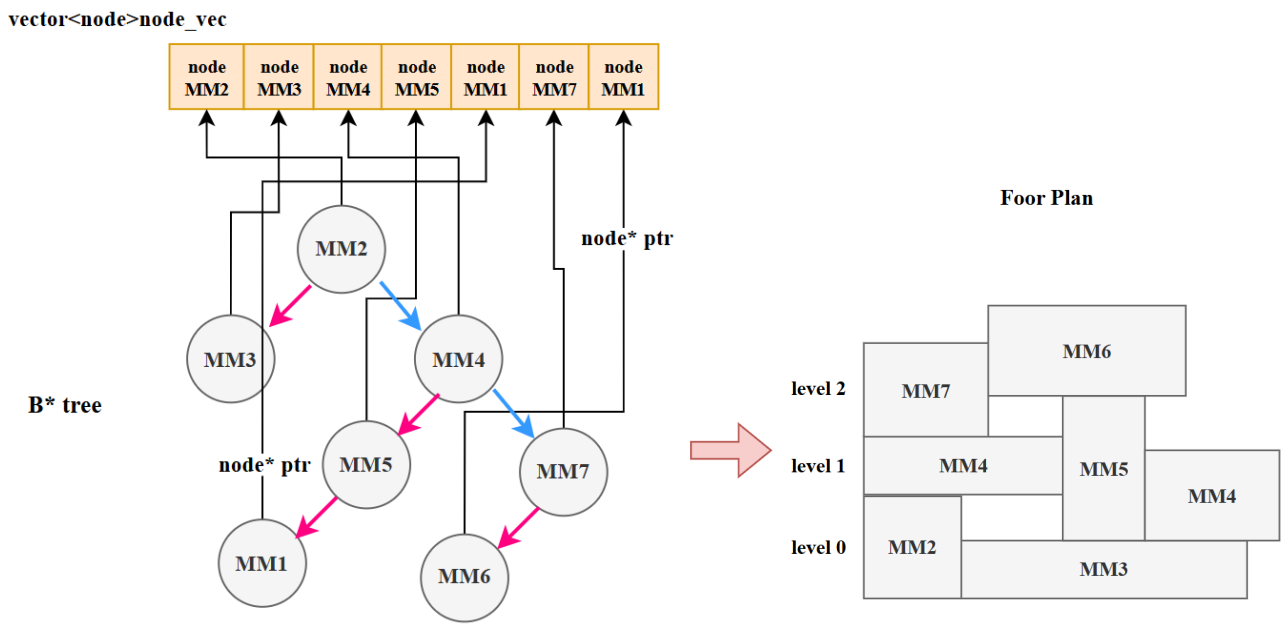
b. vector<node> node_vec

node_vec 是一個用來儲存所有 Node 實體的容器，實作上為一個 vector<node>。每個 Node 對應到一個具體的 Block，並包含該 Block 的所有關鍵資訊，如名稱、ID、位置、可能的尺寸組合等（如前述所定義），後續流程中，我們會透過建立 B*-Tree 的結構來決定每個 Block 的擺放順序與相對位置，而 node_vec 所儲存的實體則會根據這棵樹的指引，完成實際的 Floor planning 。



c. B* Tree Structure

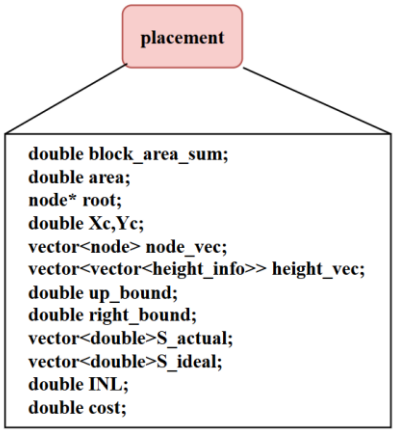
在實作上，B*-Tree 的每個節點並不儲存完整的 Block 資訊，而是僅持有指向 NodeVector 中實體的指標。如此設計除了節省記憶體空間，也使得整體架構更為清晰有條理，我們再依據這棵建好的 B*-Tree 來完成整體的 Floorplanning，作為後續優化（例如 Simulated Annealing）的初始解。



d. Placement

struct placement 是用來記錄整體 Floorplanning 最終結果的資料結構，裡面包含了最終的 bounding box 面積、bounding box 的長與寬、所有 block 的資訊（也就是 node vector），以及根據擺放結果所計算出來的 INL（Integration Non-Linearity）和此次 placement 所估計出的 cost。

```
struct placement{
    double block_area_sum;
    double area;
    node* root;                 //Btree的root
    double Xc,Yc;               //質心座標
    vector<node> node_vec;      //node vector
    vector<vector<height_info>> height_vec;
    double up_bound;            //bounded_box寬
    double right_bound;         //bounded_box長
    vector<double>S_actual;
    vector<double>S_ideal;
    double INL;
    double cost;
};
```



4. Method of My Design- Algorithm and Implementations

a. Node Vector Initialization

```
Function InitializeNodeVector(placement, input_file):
    placement.block_area_sum ← 0
    lines ← ReadFile(input_file)
    node_vec ← empty list

    For line in lines:
        node ← new Node from line header
        Initialize dimension bounds
        For each candidate in line:
            Parse candidate dimensions
            If dominated, skip
            Else if dominates all, clear candidates and reset bounds
            Else update bounds and add candidate
        placement.block_area_sum += area of first candidate
        Append node to node_vec

    Return node_vec
```

這段程式讀取區塊資訊檔案，並初始化 vector<node>，對每行資料解析出多種區塊長寬組合，過濾掉明顯不佳的候選，僅保留較佳尺寸組合作為 candidate。同時計算所有區塊的面積總和儲存於 my_placement.block_area_sum 中。

b. Sorted Block with Area

```
Function SortNodesByArea(NodeList):
    Sort NodeList in descending order by area, where
        area = width × height of the candidate at candidate_idx for each node
End Function
```

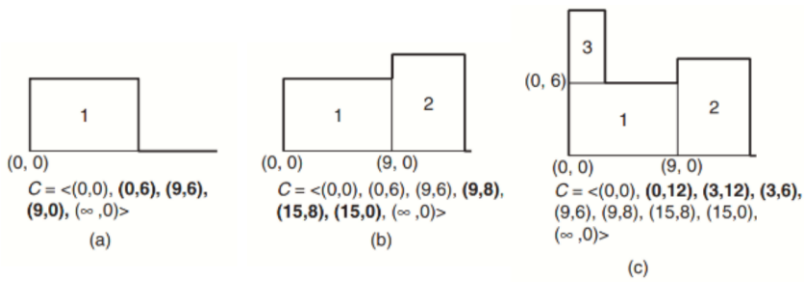
這段程式依據每個節點目前選定候選 (candidate_idx) 的長寬計算面積，並將 node_vec 依面積由大到小排序，方便後續放置較大區塊時優先考慮。

c. Building B* Tree

B*-Tree 是整體流程的核心之一，根據事先建立好的 node_vec，依照其順序來建構出整棵 B*-Tree。在建構的過程中，我們會將每個 Block 的位置同步計算出來，並繼承到對應的 Node 實體中，這個建構過程的時間複雜度為 O(N)，其中 N 為 Block 的總數。也就是說在建構 B*-Tree 的同時，就完成了所有 Block 的初始擺放位置計算。

```
Function MakeBTree(nodes, height_info, up_bound, right_bound, area_sum):
    If nodes empty, return null
    Initialize stack with (idx=0, x=0, level=0, parent_link=null), root ← null

    While stack not empty:
        Pop (idx, cur_x, level, parent_link)
        If idx ≥ nodes.size, *parent_link ← null, continue

        node ← nodes[idx]
        node.x ← cur_x
        node.y ← FindValidY(node, level, height_info)
        UpdateHeightInfo(node, level, height_info)
        up_bound ← max(up_bound, node.y + node.height)
        right_bound ← max(right_bound, node.x + node.width)
        *parent_link ← node; node.left ← null; node.right ← null

        If cur_x + node.width ≥ sqrt(area_sum):
            Push (idx+1, 0, level+1, node.right)
        Else:
            Push (idx+1, cur_x + node.width, level, node.left)

    Return root
```

d. Contour Maintenance



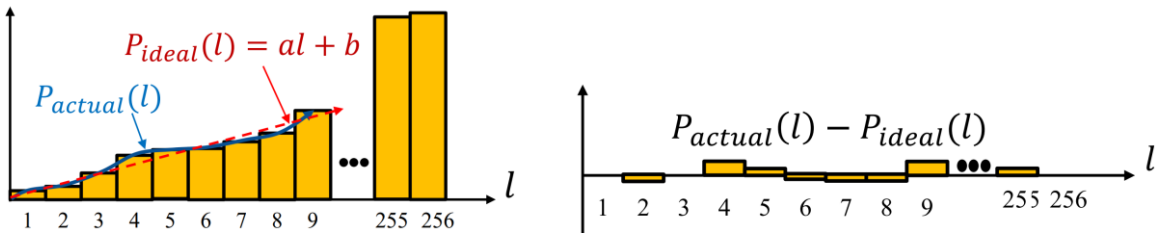
C = 1　　C = 1, 2　　C = 3, 1, 2

```
void update_height_vec(node cur_node, int lvl, vector<vector<height_info>>& hvec) {
    int idx = cur_node.candidate_idx;
    height_info info = {
        cur_node.x,
        cur_node.x + cur_node.candidate[idx].width,
        cur_node.y + cur_node.candidate[idx].height
    };
    if (lvl == hvec.size()) hvec.push_back({info});
    else hvec[lvl].push_back(info);
}
```

這段程式將目前放置的節點資訊新增到對應層級的 height_vec 中。若當前層級尚未建立，則新增一個新的子向量；否則直接加入對應層的 height_info，用來記錄該層水平區段的高度資訊。

e. INL Calculation

$$INL = \max_{l} |P_{actual}(l) - P_{ideal}(l)|$$




```
Function Make_S_actual(placement):
    sorted_vec ← Sort_nodes_by_id(placement.node_vec)
    N ← length(sorted_vec)
    S_actual.resize(N)
    Xc ← placement.Xc
    Yc ← placement.Yc
    S_actual[0] ← (sorted_vec[0].xk - Xc)^2 + (sorted_vec[0].yk - Yc)^2
    For i in 1 to N-1:
        S_actual[i] ← S_actual[i-1] + (sorted_vec[i].xk - Xc)^2 + (sorted_vec[i].yk - Yc)^2
```

```
Function Make_S_ideal(placement):
    N ← length(S_actual)
    sum_x, sum_y, sum_x_sqr, sum_xy ← 0
    For i in 0 to N-1:
        sum_x ← sum_x + i
        sum_y ← sum_y + S_actual[i]
        sum_x_sqr ← sum_x_sqr + i * i
        sum_xy ← sum_xy + i * S_actual[i]
    a ← (N * sum_xy - sum_x * sum_y) / (N * sum_x_sqr - sum_x * sum_x)
    b ← (sum_y - a * sum_x) / N
    Resize S_ideal to N
    For i in 0 to N-1:
        S_ideal[i] ← a * i + b
```

```
Function Find_INL(placement):
    Make_S_actual(placement)
    Make_S_ideal(placement)
    INL ← 0
    For i in 0 to length(S_actual) - 1:
        INL ← max(INL, abs(S_actual[i] - S_ideal[i]))
```

這段程式計算實際區塊分布與理想分布的最大差異（INL）。先產生實際與理想的分布 S_actual 和 S_ideal，再遍歷每個位置，取兩者絕對差的最大值作為 INL，評估 placement 品質是否平衡。

f. Simulated Annealing

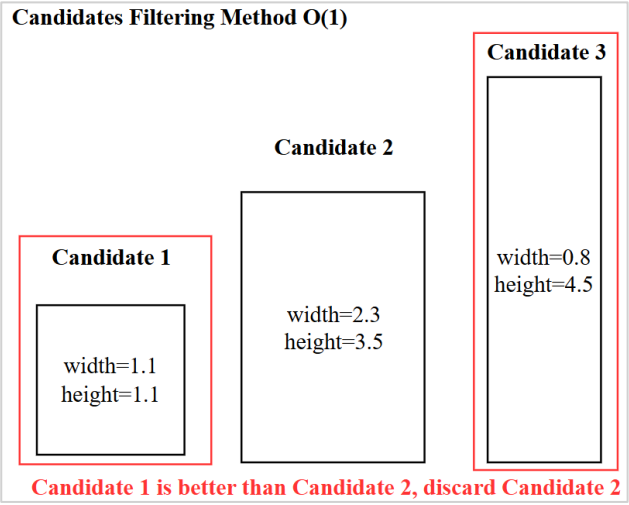
```
placement SA_placement(placement &p, string fname){
    placement cur; copy_placement(p, cur);
    double t=1000, c=0.99;
    for(int i=0; i<10000; i++){
        placement nxt; copy_placement(cur, nxt);
        Perturb_node_vec(nxt);
        clean_placement(nxt);
        nxt.root=make_Btree(nxt.node_vec,nxt.height_vec,nxt.up_bound,nxt.right_bound,nxt.block_area
        find_centroid(nxt);
        find_INL(nxt);
        nxt.cost=find_cost(nxt);
        double d=nxt.cost-cur.cost;
        if(d<0 || exp(-d/t) > (double)rand()/RAND_MAX) copy_placement(nxt, cur);
        t*=c;
        if(i%500==0 && i>0) write_file(cur, fname);
    }
    return cur;
}
```

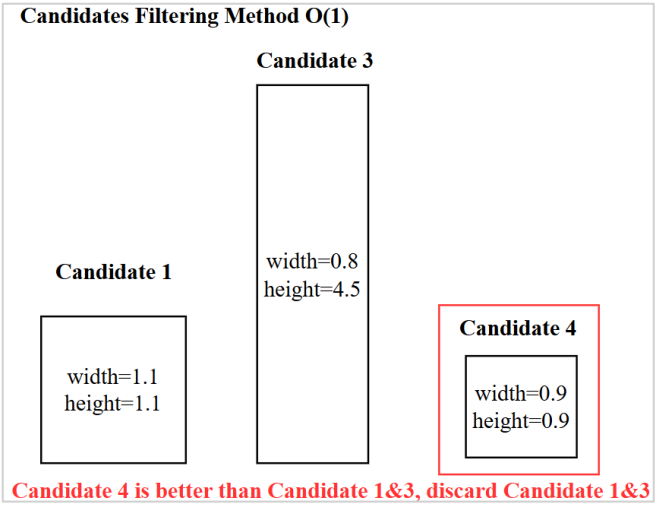
這段程式使用 Simulated Annealing 演算法優化 floorplan，起始從 my_placement 複製為 cur_placement，每輪迭代隨機改變節點順序或形狀產生 new_placement，計算其成本（cost）。若成本較低或符合機率接受條件，則更新 cur_placement。溫度隨迭代下降，並定期將最佳結果寫入檔案。最終回傳優化後的放置結果。

5. Algorithm Details- Block Size Filtering

在讀取資料的階段，我們會先對一些明顯表現不佳的 block size 進行篩選。其原理與課堂上教授所講的方法一致：只要發現某一個 block 在長度與寬度兩個維度上，同時優於另一個特定的 block，則該特定 block 將會直接從候選解（candidate vector）中移除。

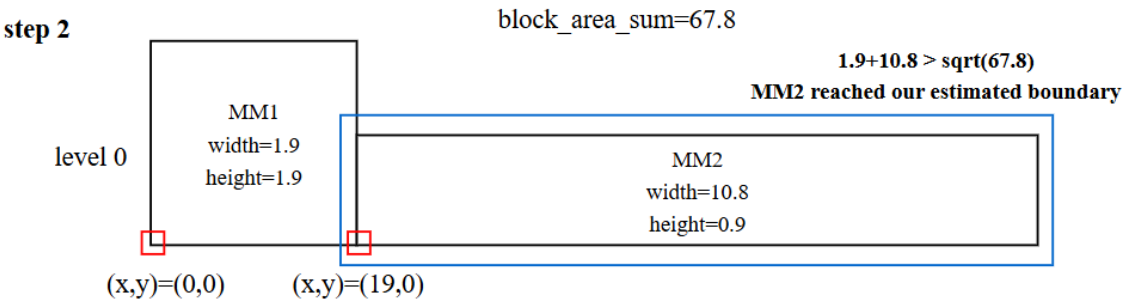
透過這樣的篩選機制，可以有效縮減解空間，避免冗餘且無效的解被納入考量，進而加速整個模擬退火（Simulated Annealing, SA）流程的運算效率。

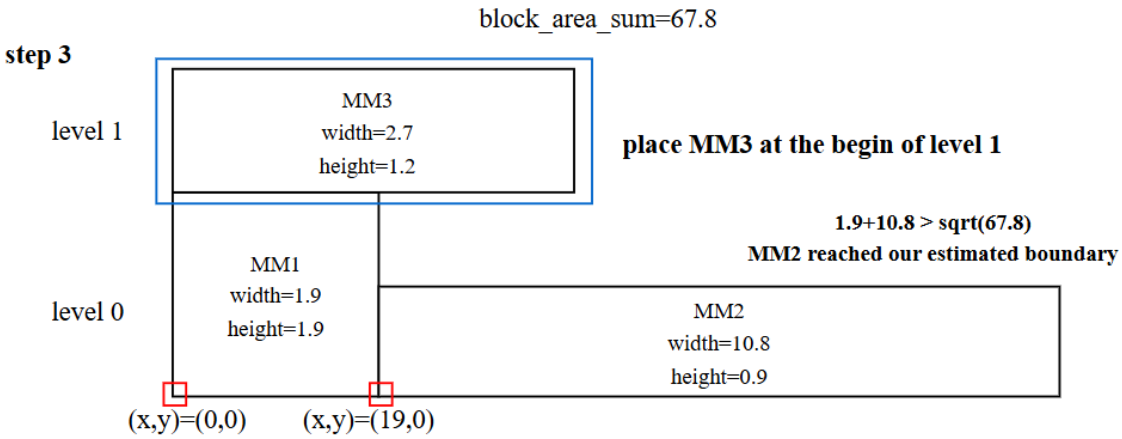




6. Algorithm Details- Block Placement Logic

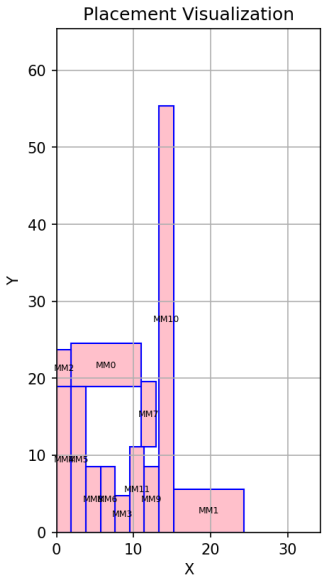
在 B*-Tree 的擺放邏輯中，為了避免長寬比過於懸殊，我們採用特定的策略來控制這個問題。首先，我們會將所有待擺放的 block 面積進行加總，計算出總面積。在擺放過程中，如果我們發現當前 block 擺放後的寬度，已經超過了這個總面積開根號的值，那麼我們就推斷下一個 block 的位置，不應該繼續向右延伸擺放，而是應該往更上一層的 level 進行擺放，藉此有效避免 block 的長寬比出現過於極端的狀況，像是過於寬扁或過於細長的情況。



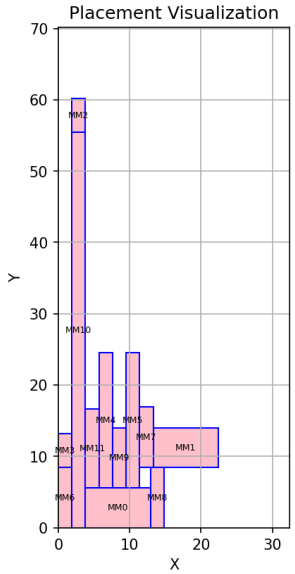


7. Algorithm Details- Block Sorting

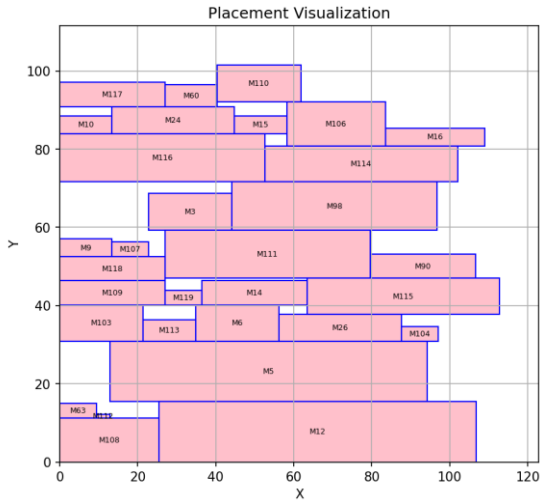
在產生初始解之前，我們會先依照 block 的面積對其進行排序。這樣做的原因，是為了盡量避免後面擺放的 block 面積過大，對整個 floorplan 的形狀造成過於劇烈的影響。將較大的 block 優先擺放，經過實驗也證明能獲得明顯較好的結果。



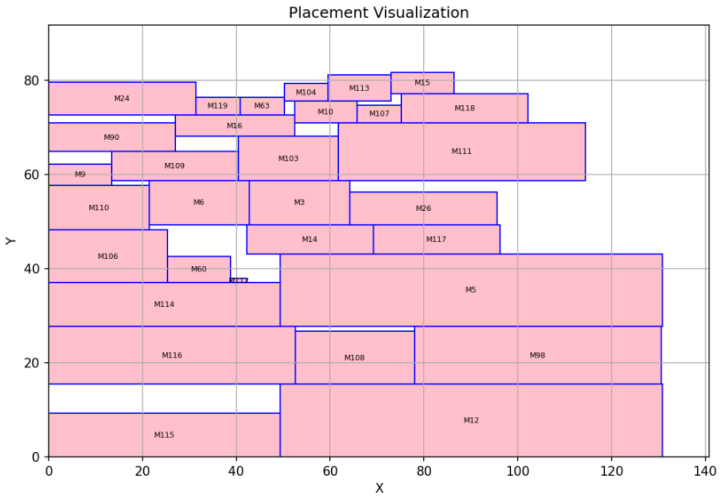
without sorting block in advance



with sorting block in advance



without sorting block in advance

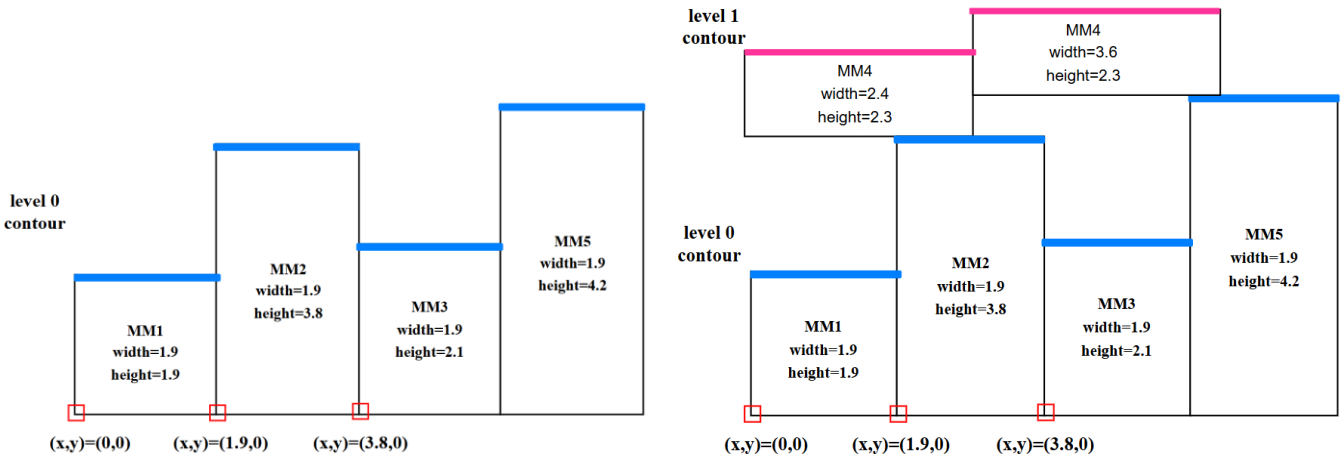


with sorting block in advance

8. Algorithm Details- Contour Logic

在維護我們的 contour 時，我們主要維護的是 horizontal contour，我們只需要記錄每個 level 上 contour 的高度以及對應的 x 座標。透過這樣的紀錄，我們可以成功在每個 level 把 block 擺放到正確的位置，並且達到 left-bottom compact 的效果。

實作上，我們和課堂中所教的方法相同，會維護一個 vector 來實際記錄 contour 的狀態，在邏輯上，我們會記錄一個 block 擺放的橫坐標範圍，並取該範圍內 contour 的最大高度作為該 block 擺放的 y 座標下限，以確保這個 block 不會和已有的 block 重疊。



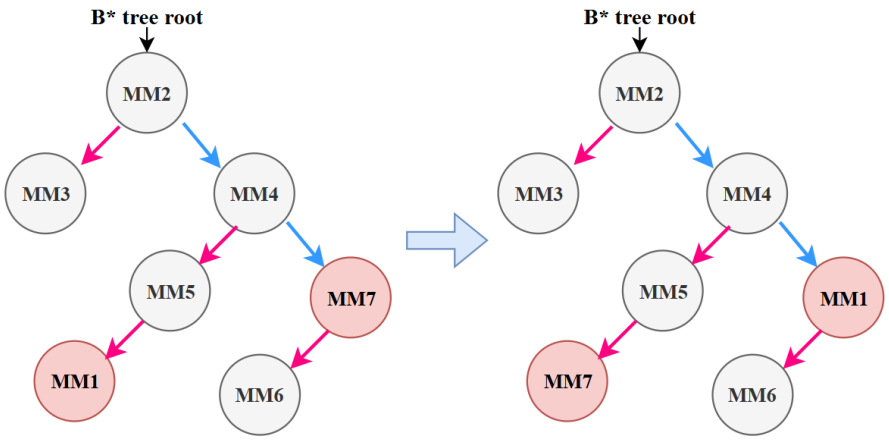
9. Algorithm Details- Perturbation Strategy

```
void Perturb_node_vec(placement &cur) {
    if (rand() % 100 < 90)
        swap_two_random_node(cur.node_vec);
    else
        resize_random_node(cur);
}
```

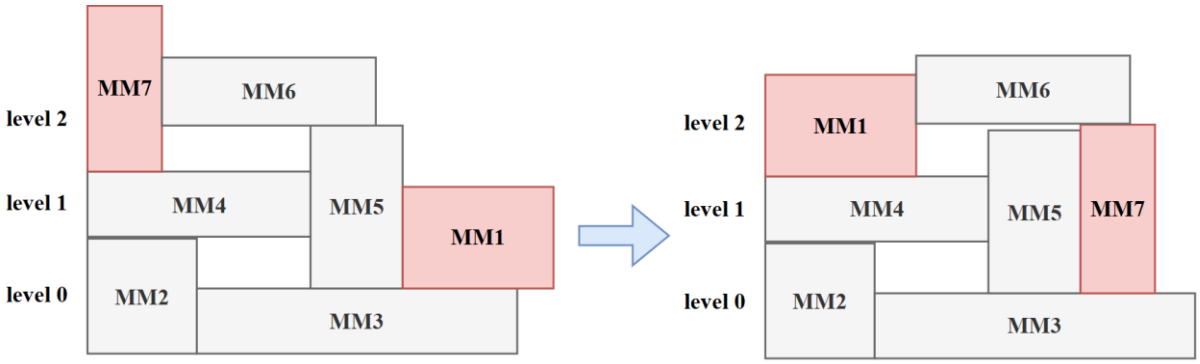
Perturbation Strategy 的部分，在每一次 Perturbation 中，我們有 90%的機率會選取兩個 Node 進行交換，另外 10%的機率會改變 Block 的長寬，在改變 Block 長寬時，我們會先確認該 Block 是否有多組長寬選項，只有當 Block 有不止一組長寬時，才會執行 Change Shape。若發現 Block 沒有多組長寬，我們會額外嘗試五次，若五次嘗試後仍未找到可更改長寬的 Block，則放棄本次 Perturbation。

a. Randomly Swap 2 Block

```
void swap_two_random_node(vector<node> &node_vec) {
    int n = node_vec.size();
    if (n < 2) return;

    int i = rand() % n;
    int j = (i + 1 + rand() % (n - 1)) % n;  // 保證 j ≠ i

    swap(node_vec[i], node_vec[j]);
}
```

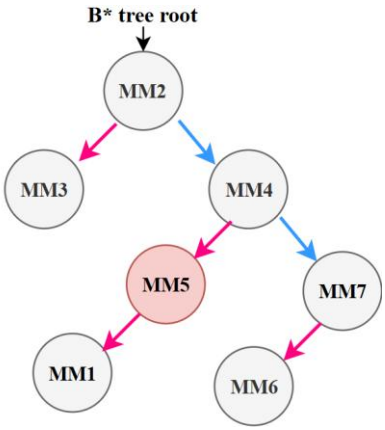


Randomly Swap 2 Block

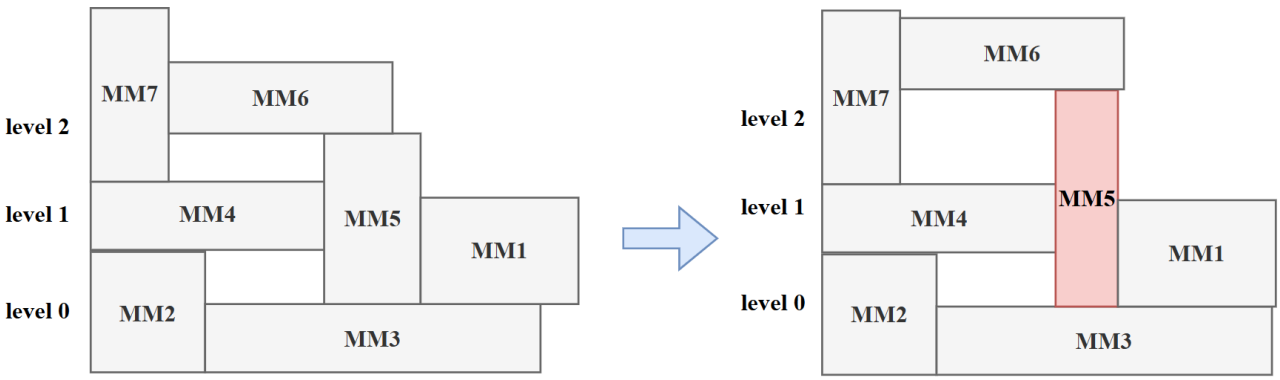


Randomly Swap 2 Block

b. Change Shape of Random Block




```
void resize_random_node(placement &p){
    int n = p.node_vec.size();
    for(int i = 0; i < 5; i++){
        int idx = rand() % n;
        auto &nd = p.node_vec[idx];
        if(nd.candidate.size() > 1){
            int old_idx = nd.candidate_idx, new_idx;
            do { new_idx = rand() % nd.candidate.size(); } while(new_idx == old_idx);
            p.block_area_sum -= nd.candidate[old_idx].width * nd.candidate[old_idx].height;
            p.block_area_sum += nd.candidate[new_idx].width * nd.candidate[new_idx].height;
            nd.candidate_idx = new_idx;
            return;
        }
    }
}
```



Change Shape of Random Block

10. Algorithm Details- Cost Function Design

```
double find_cost(const placement &p){
    double AR = p.up_bound / p.right_bound;
    if (AR < 1) AR = 1 / AR;  // 取大於等於1的比例

    double F_AR = 0.0;
    if (AR > 2) F_AR = AR - 2;
    else if (AR < 0.5) F_AR = 2 * (AR - 0.5);

    return p.up_bound * p.right_bound * (1 + F_AR) + 8 * p.INL;
}
```

Cost function 的部分，因為在 simulated annealing 的過程中，我們需要針對 cost 計算 delta 值，進而決定被接受的機率，因此必須設計 cost function。我們完全依照文件上的定義，但因為 INL 和面積資訊需要同時考慮，所以設定面積貢獻的 cost 加上 8 倍的 INL，這個 8 是目前測試後認為相對較佳的權重。

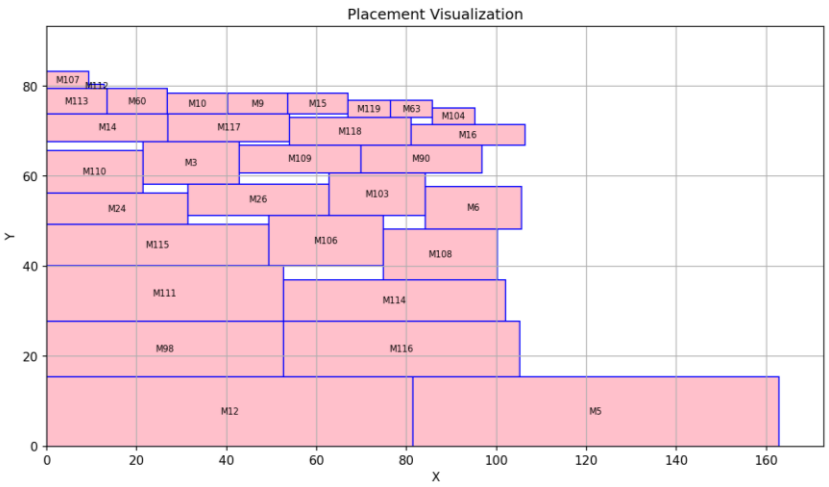
11. Algorithm Details- SA Details

Simulated Annealing 的部分，我們設定總共進行一萬次 Iteration，每一次的 cooling rate 為 0.99，初始溫度設定為 1000 度。這組參數是目前測試後，結果相對較佳的設定，以下比較 Floorplan Results 在執行 SA 以前與以後的結果:

a. Case2.block:

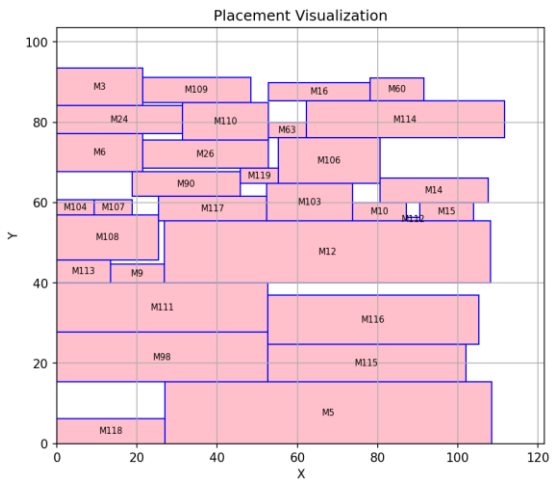
Without SA

Area= 13551.4720
(width,height)= (162.80, 83.24)
INL=6062.46



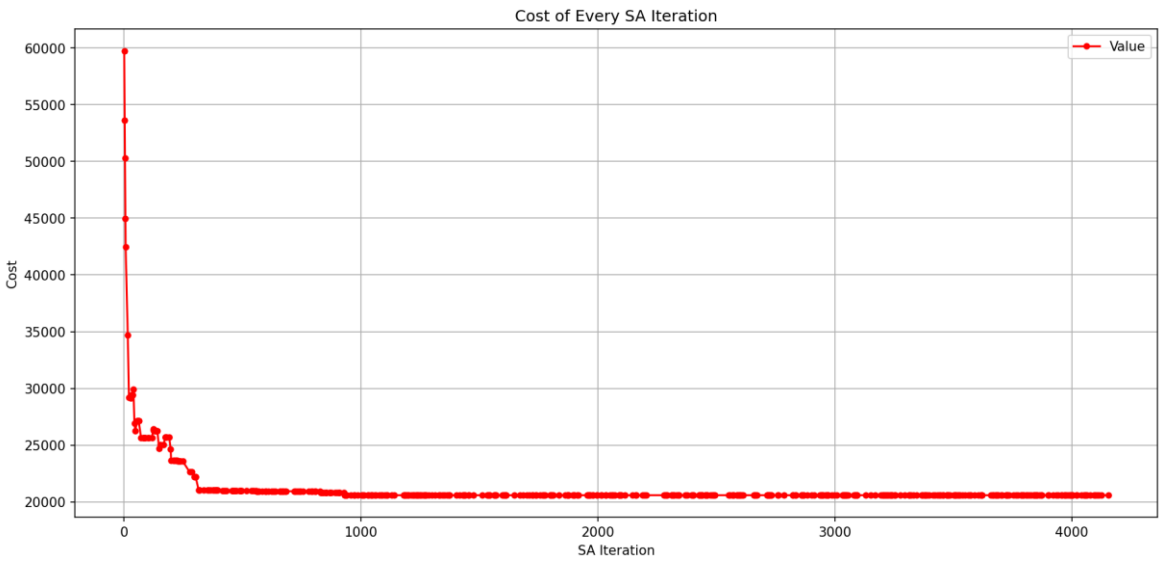
After SA

Area= 10436.8320
(width,height)= (111.60, 93.52)
INL=1259.95



說明: 可以看出我們 SA 的效果非常卓越，INL 降低了 80%，而 Area 也縮小 22.9%

SA Process Cost Evolution of Case2.block

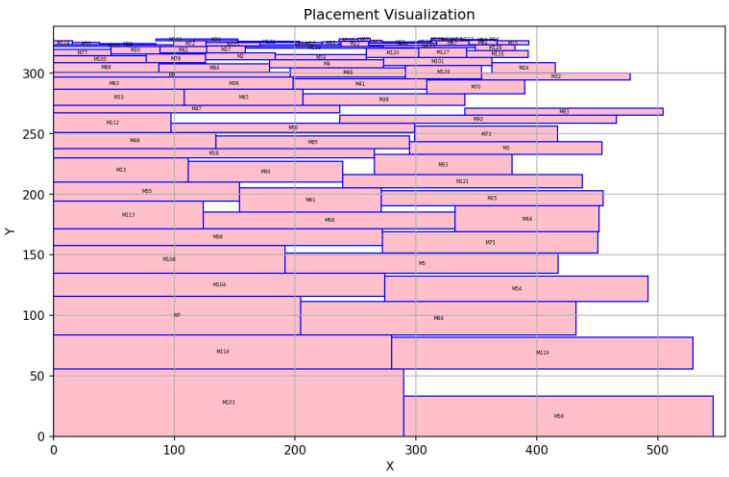


說明:以此可見，以 Case2.block 而言，SA iteration 到約 500 次之後就沒有顯著降低 cost

b. Case3.block (My Self-Generated Case)

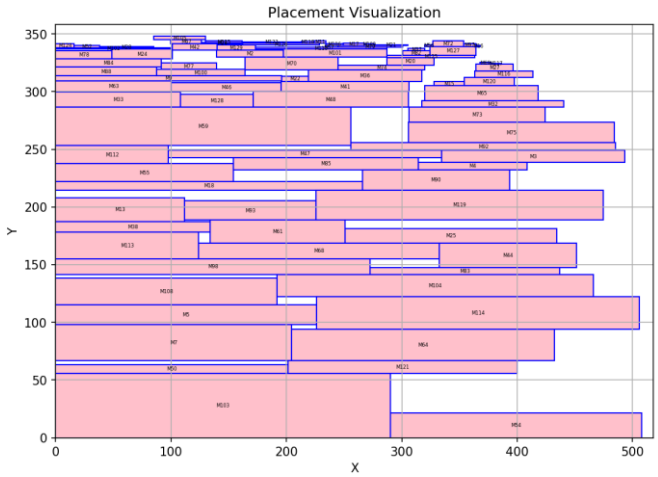
Without SA

Area= 179471.3138
(width,height)=(546.02, 328.69)
INL=75660.21

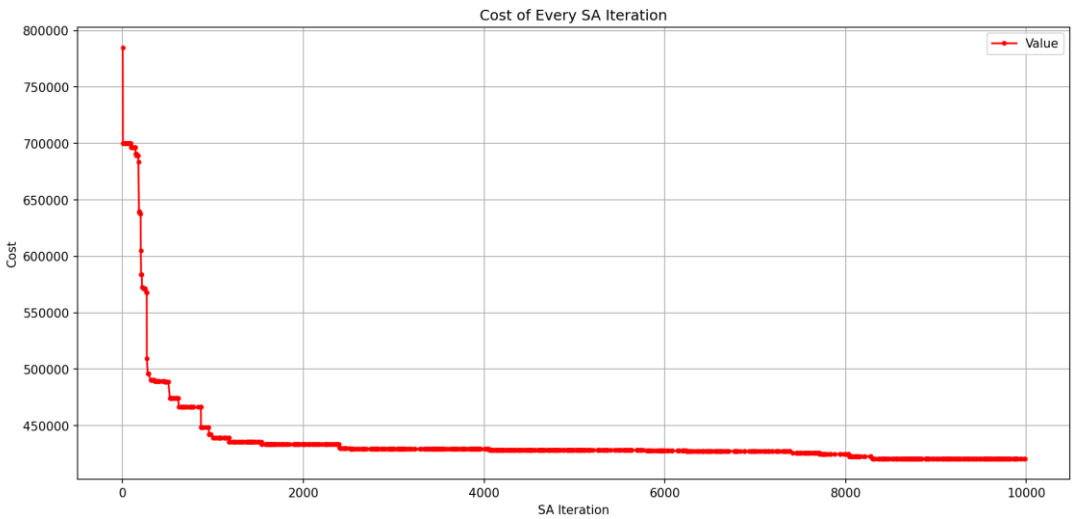


After SA

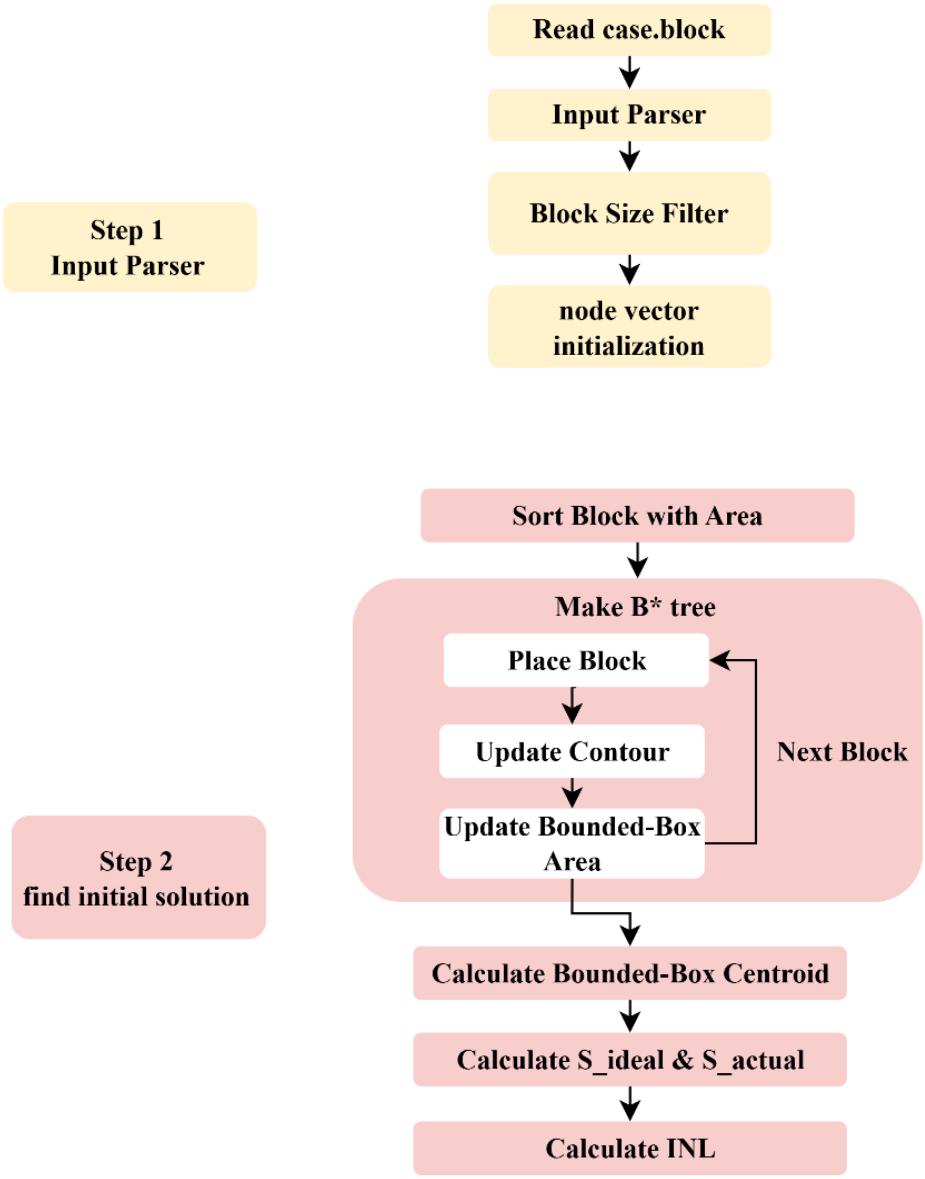
Area= 176729.0618
(width,height)=(507.74, 348.07)
INL=35540.79

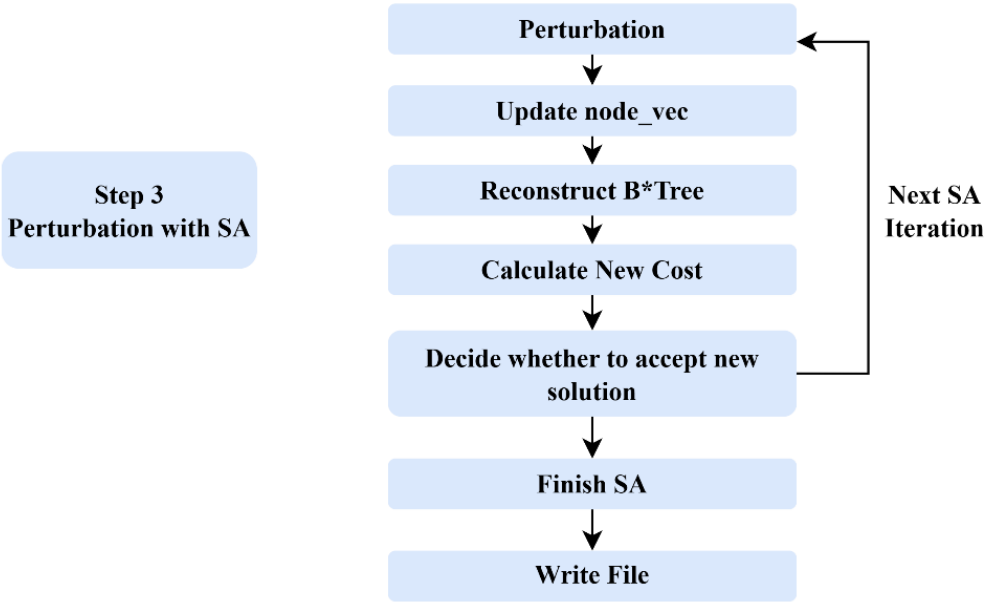


說明: 可以看出我們 SA 的效果非常卓越，INL 降低了約 50%，而 Area 也成功縮小，以 case3.block 而言，SA iteration 到約 2500 次之後就沒有顯著降低 cost



12. Flow Chart of My Program



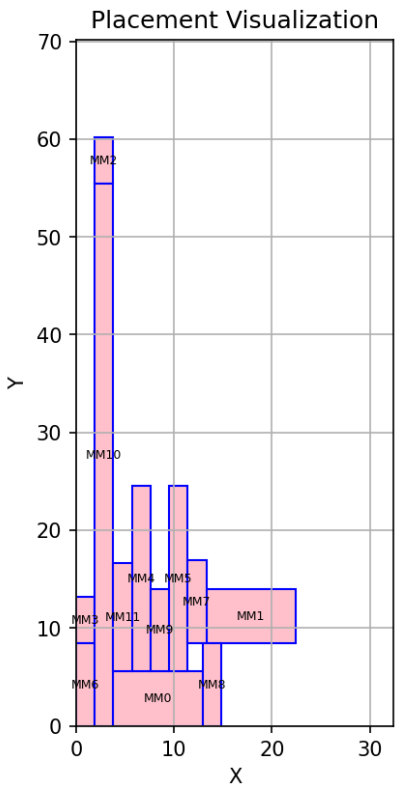


以上為 PA4 我實作上的 Program Flow Chart

13. Test Cases Results

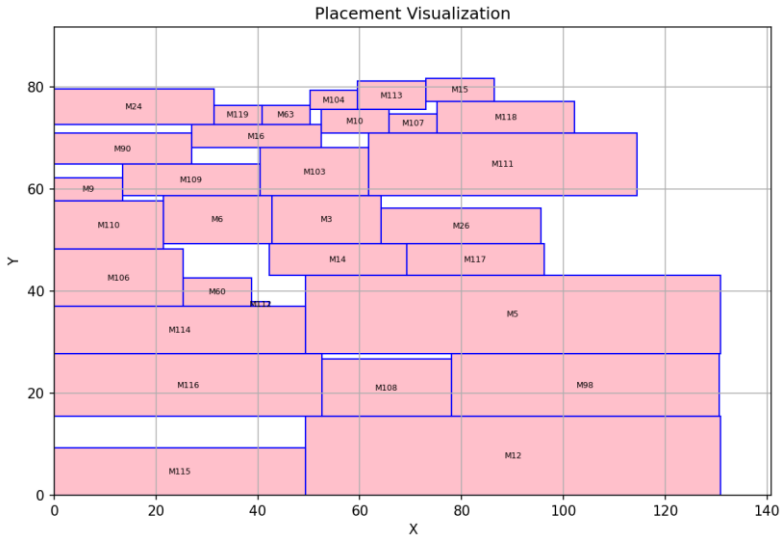
a. Public case 1

Width, Height= (22.41 60.94)



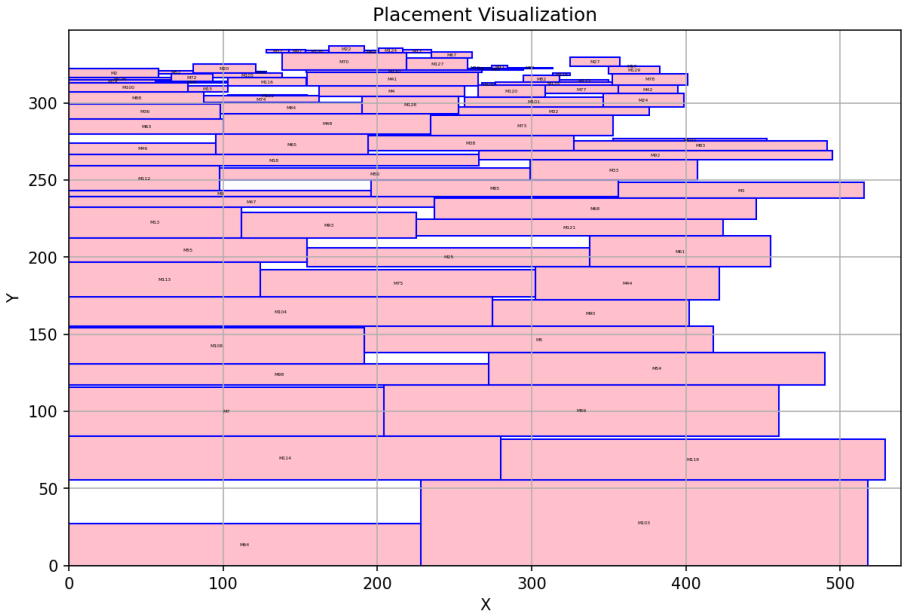
b. Public Case 2

Width, Height= (108.40, 95.44)



c. My self-generated Case

Width, Height= (529.32, 337.12)



14. The Hardness of This Assignment

這次 PA 中，我覺得最大的困難是對 B* Tree 這個資料結構的熟悉度。因為對我來說，B* Tree 是比較新的東西，也不太確定有哪些策略比較適合這次的 PA。所以很幸運能有蔡昀翰的幫助，讓我比較快掌握建構的方式。真的非常感謝昀翰，沒有他的幫忙，我在 PA4 上面會更痛苦，那種痛苦主要來自於沒有方向的迷茫感。

最後，也希望助教在期末考和 final project 的評分上可以稍微寬鬆一點。因為說實在的，我現在真的是差點就要 fail 這門課了，這對我來說真的太瘋狂了，畢竟我在這門課花了非常多時間努力。

15. Suggestions About PA4

目前沒太多建議，但依照其他同學的建議，如果 report 給分可以再更寬一點的話應該會更好，在上次 PA3 有些同學 report 做了超過 20 頁，但分數上依然沒有很理想，且其他同學反應 report 給分稍微有點嚴格

這次報告我花了整整兩天的時間準備，已經把我能寫的內容都完整呈現了。希望不要太低分，不然我要被當了…**最後也祝助教期末考順利，研究與求職都一切順利，謝謝！**

16. Reference

a. **Hu, R., Huang, Z., Tang, Y., Van Kaick, O., Zhang, H., & Huang, H. (2020). Graph2plan: Learning floorplan generation from layout graphs. *ACM Transactions on Graphics (TOG)*, *39*(4), 118-1.**

b. **Otten, R. H. (1982, June). Automatic floorplan design. In *19th Design Automation Conference* (pp. 261-267). IEEE.**

c. **Michalek, J., Choudhary, R., & Papalambros, P. (2002). Architectural layout design optimization. *Engineering optimization*, *34*(5), 461-484.**